Introduction to Deep Learning for Text Processing

Carolina Scarton
c.scarton@sheffield.ac.uk

Text Processing
Department of Computer Science, University of Sheffield
Autumn Semester 2023

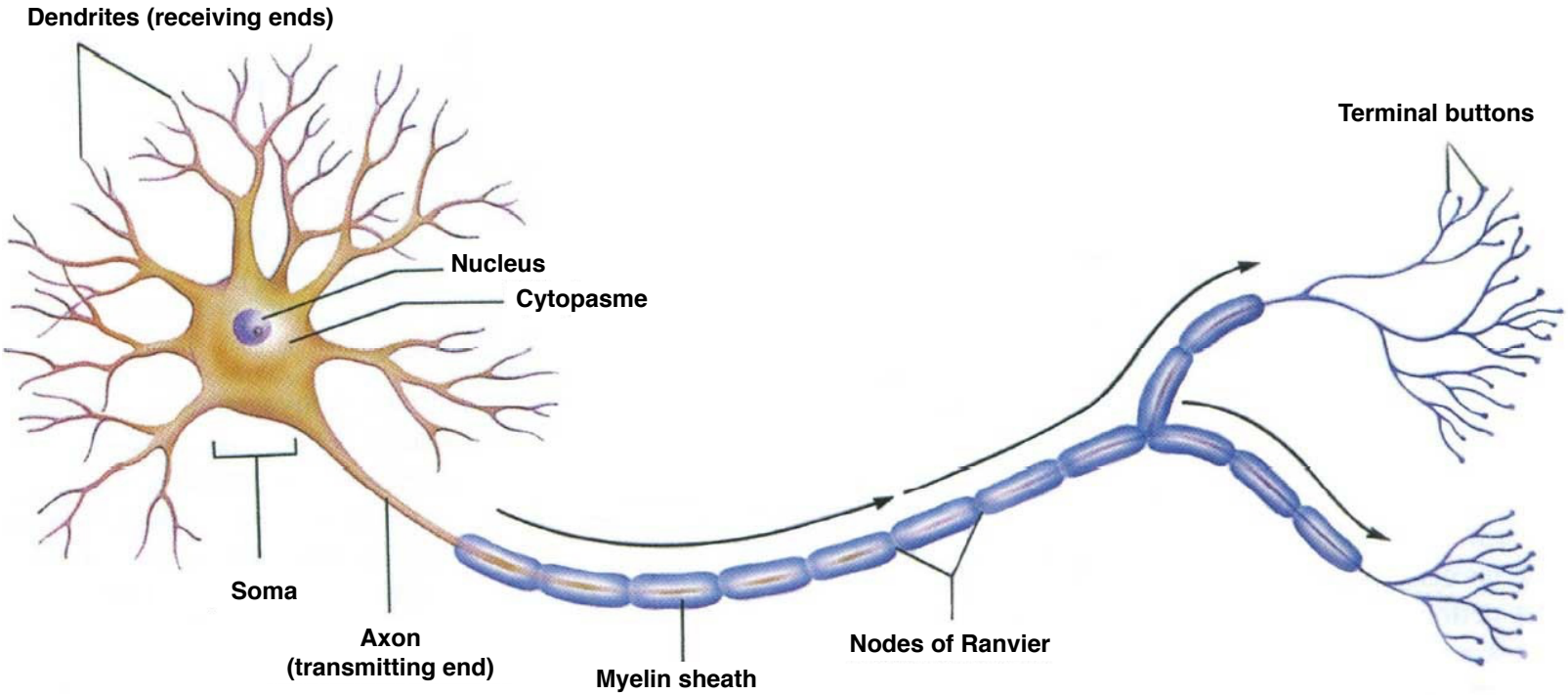
Learning Objectives

- Explain the skip-gram model for word embeddings
- Name some of the applications of word embeddings
- Name techniques for sentence representation

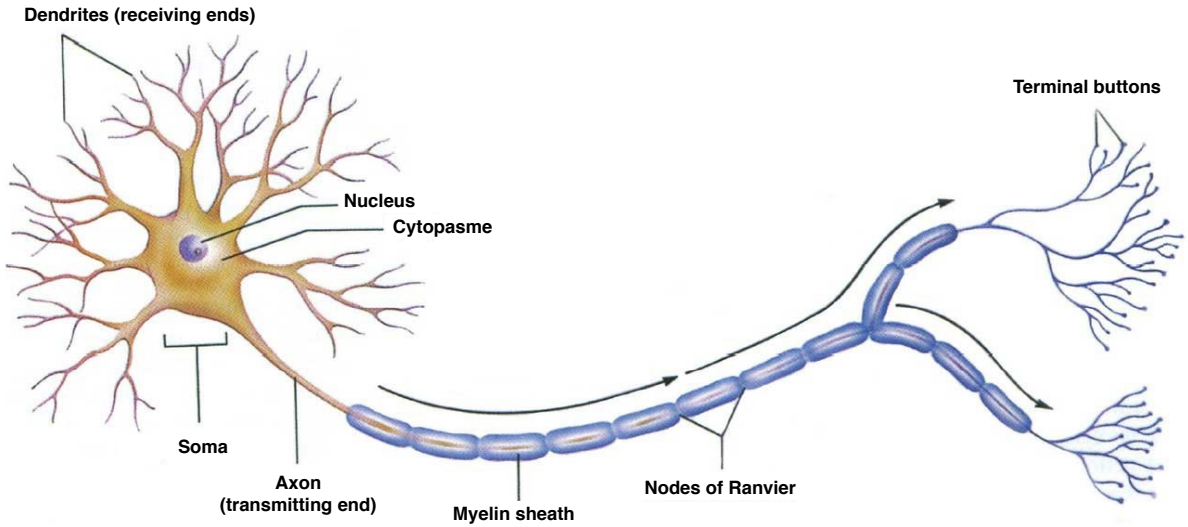
Text Processing: Deep Learning: Overview

- **Shortest introduction to Neural networks**
- Representing words
- Representing sentences
- Deep Learning for Sentiment Analysis
- Deep Learning for Information Extraction (NER)

Biological neuron / nerve cell

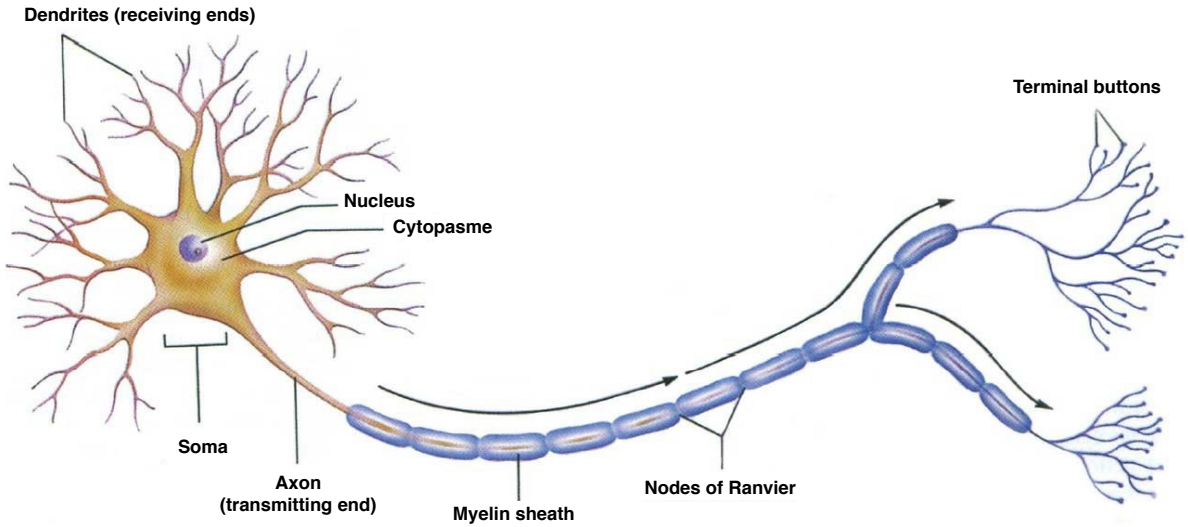


Hebb principle



Hebb: **"Neurons that fire together, wire together"**

Hebb principle



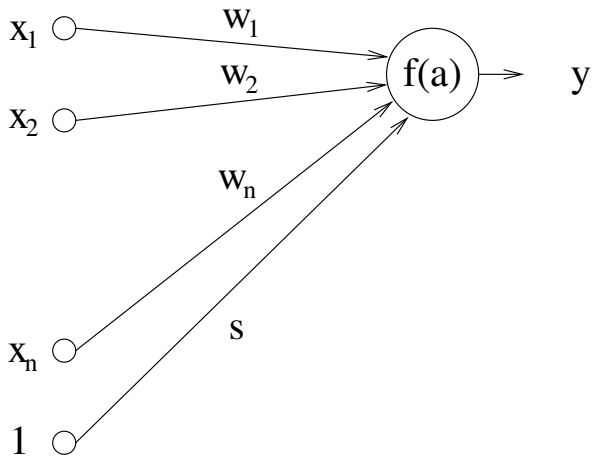
Hebb: **"Neurons that fire together, wire together"**

- Cells are active together $\rightarrow$ reinforce their connection
- Cells are not active together $\rightarrow$ diminish their connection

$\Rightarrow$ **local process** there is no global supervision

The perceptron

Perceptron: computing unit loosely inspired by the biological neuron

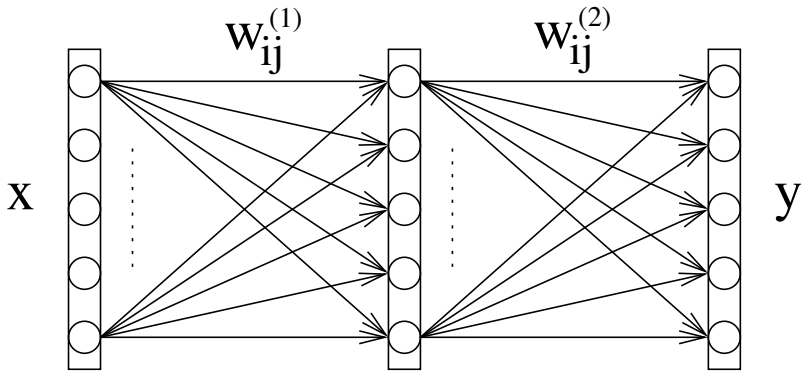


input:	$\mathbf{x} = \{x_i\}$
weights:	$\mathbf{w} = \{w_i\}$
threshold:	s
activity:	$a = \sum_i w_i x_i + s$
activation function:	$f =$ threshold function
output:	$\hat{y} = f(a)$

Training method: change the weights $\mathbf{w}$ if a training example $\mathbf{x}$ is misclassified as follows:

$$\mathbf{w}^{new} = \mathbf{w}^{cur} + \mathbf{x}.y \qquad \text{with} \qquad y \in \{+1, -1\}$$

Multilayer perceptron



$$
\begin{aligned}
y_i^2 &= f\left(\sum_j w_{ij}^1 \; x_j^1\right) \\
y_i^3 &= f\left(\sum_j w_{ij}^2 \; y_j^2\right) \\
&\vdots \\
y_i^c &= f\left(\sum_j w_{ij}^{c-1} \; y_j^{c-1}\right)
\end{aligned}
$$

$\Rightarrow$ **propagation** of the input **x** towards the output **y**

Multilayer perceptron: training

1. Normalise data
2. Initialise the weights $\mathbf{W}$
3. Repeat
   - Pick a **batch** of examples $(\mathbf{x}, \mathbf{c})$
   - **Forward** pass: propagate the batch $\mathbf{x}$ through the network $\rightarrow \mathbf{y}$
   - Calculate the error $E(\mathbf{y}, \mathbf{c})$
   - **Backward** pass: **backpropagation** $\rightarrow \nabla w_{ij}$
   - Update weights $w_{ij}^{new} = w_{ij}^{cur} - \lambda \frac{\partial E}{\partial w_{ij}}$
   - Eventually change the training meta-parameters (e.g. learning rate λ)

   until convergence

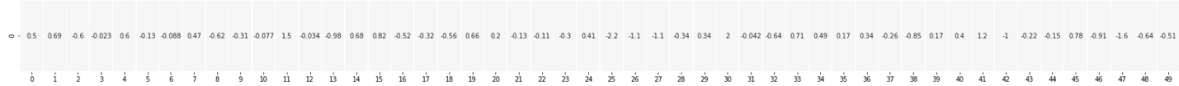
That's great, but where is the text?!?

Text Processing: Deep Learning: Overview

- Shortest introduction to Neural networks
- **Representing words**
- Representing sentences
- Deep Learning for Sentiment Analysis
- Deep Learning for Information Extraction (NER)

How to represent words?

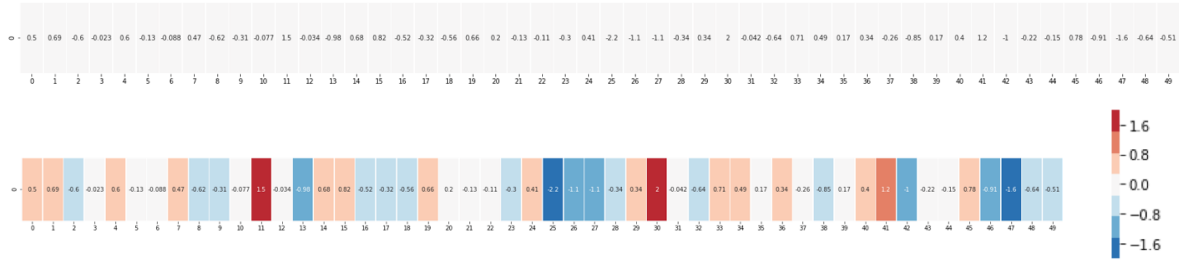
The semantic properties of the words are encoded in the dimensions of the vector



[Source: `http://jalammar.github.io/illustrated-word2vec/` ← **Must read!**]

How to represent words?

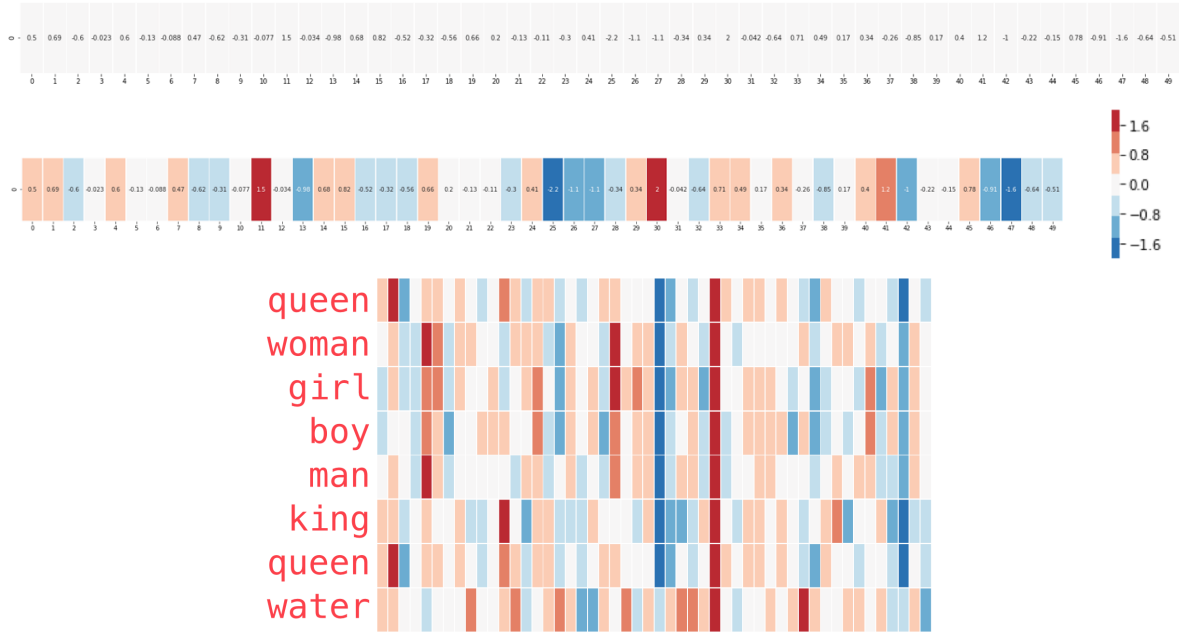
The semantic properties of the words are encoded in the dimensions of the vector



[Source: `http://jalammar.github.io/illustrated-word2vec/` ← **Must read!**]

How to represent words?

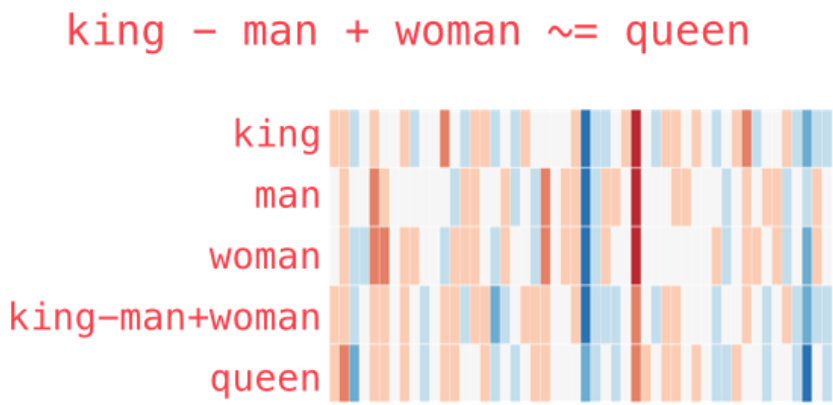
The semantic properties of the words are encoded in the dimensions of the vector



[Source: `http://jalammar.github.io/illustrated-word2vec/` ← **Must read!**]

How to represent words?

The semantic properties of the words are encoded in the dimensions of the vector



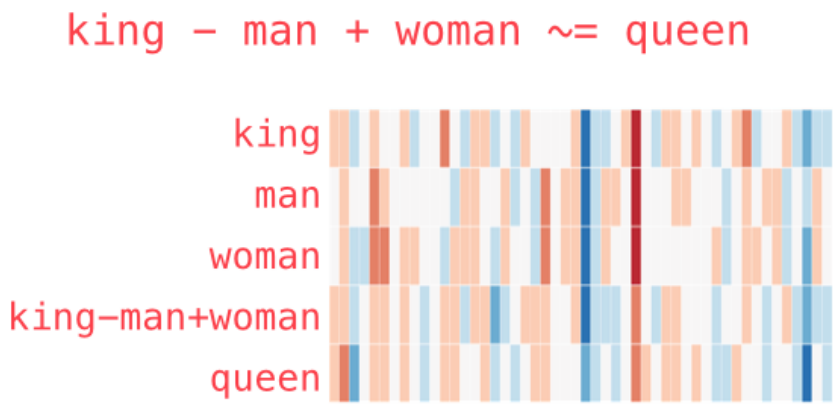
Can be learned in several ways:

- Extract handcrafted meaningful features
- Use a neural network!

[Source: `http://jalammar.github.io/illustrated-word2vec/` ← **Must read!**]

How to represent words?

The semantic properties of the words are encoded in the dimensions of the vector



Can be learned in several ways:

- Extract handcrafted meaningful features
- **Use a neural network!**

[Source: `http://jalammar.github.io/illustrated-word2vec/` ← **Must read!**]

How to represent words?

Word Embeddings

Vector representation of a word $\Rightarrow$ vector of real values

Also called continuous space representation.

How to represent words?

Word Embeddings

Vector representation of a word $\Rightarrow$ vector of real values

Also called continuous space representation.

- What would be the simplest way of obtaining vectors?

How to represent words?

Word Embeddings

Vector representation of a word $\Rightarrow$ vector of real values

Also called continuous space representation.

- What would be the simplest way of obtaining vectors? $\Rightarrow$ so-called **1-hot vector**:
  - vector of size equal to **vocabulary size**
  - contains 0 everywhere except for a single 1 at a specific position

a	=	[1,0,0,0,0,0,...,0]
an	=	[0,1,0,0,0,0,...,0]
bowl	=	[0,0,1,0,0,0,...,0]
cat	=	[0,0,0,1,0,0,...,0]
...	...	...
Zyzzyva	=	[0,0,0,0,0,0,...,1]

How to represent words?

Word Embeddings

Vector representation of a word $\Rightarrow$ vector of real values

Also called continuous space representation.

- What would be the simplest way of obtaining vectors? $\Rightarrow$ so-called **1-hot vector**:
  - vector of size equal to **vocabulary size**
  - contains 0 everywhere except for a single 1 at a specific position

a	=	[1,0,0,0,0,0,...,0]
an	=	[0,1,0,0,0,0,...,0]
bowl	=	[0,0,1,0,0,0,...,0]
cat	=	[0,0,0,1,0,0,...,0]
...	...	...
Zyzzyva	=	[0,0,0,0,0,0,...,1]

- Is that a good representation?

How to represent words?

Word Embeddings

Vector representation of a word ⇒ vector of real values

Also called continuous space representation.

- What would be the simplest way of obtaining vectors? ⇒ so-called **1-hot vector**:
  - vector of size equal to **vocabulary size**
  - contains 0 everywhere except for a single 1 at a specific position

a	=	[1,0,0,0,0,0,...,0]
an	=	[0,1,0,0,0,0,...,0]
bowl	=	[0,0,1,0,0,0,...,0]
cat	=	[0,0,0,1,0,0,...,0]
...	...	...
Zyzzyva	=	[0,0,0,0,0,0,...,1]

- Is that a good representation? ⇒ **NO!**
  - distance between any two words is the same for all word pairs
  - position of the "1" arbitrarily
  - → it is just a **coding**

Previous classes

- **Count-based distributional vectors**
  - Count number of times a **word co-occurs with other words**
  - TFIDF, LSA, LDA, etc

Previous classes

- **Count-based distributional vectors**
  - Count number of times a **word co-occurs with other words**
  - TFIDF, LSA, LDA, etc
- Problems:
  - Vector size **increases with vocabulary**
  - **High dimensionality**

Previous classes

- **Count-based distributional vectors**
  - Count number of times a **word co-occurs with other words**
  - TFIDF, LSA, LDA, etc
- Problems:
  - Vector size **increases with vocabulary**
  - **High dimensionality**
- **SVD** $\rightarrow$ solve dimensionality issues, but
  - **Computational costs** is high $\rightarrow$ not feasible for millions of words or documents
  - **Difficult to add new words** to the model

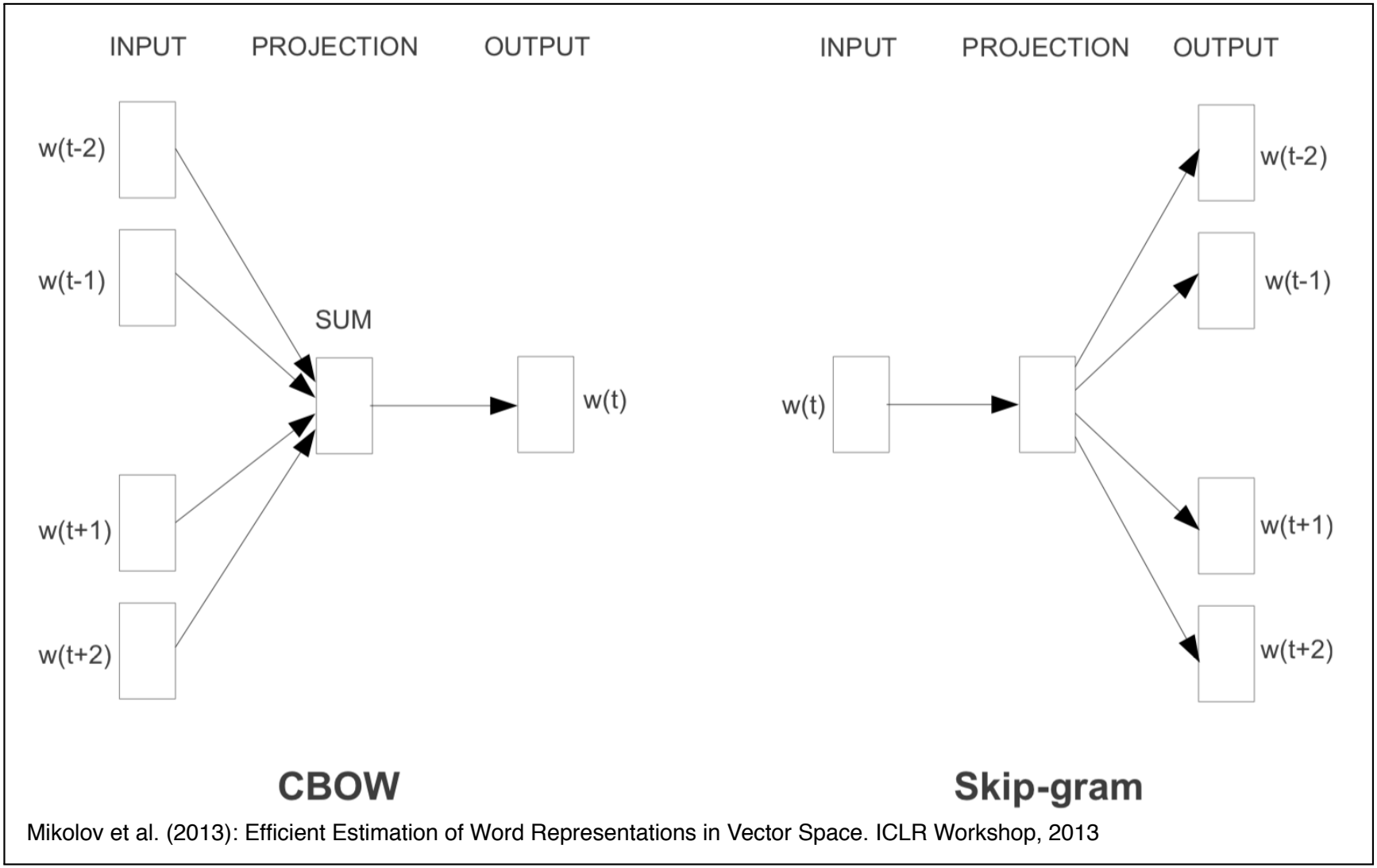
Prediction-based distributional vectors

- **Neural network** algorithms
- Directly build **dense, low-dimensional** representations
- **Embeddings:** words are **embedded** in a low dimensional vector

Predict between **every word** and its **context words**

word2vec

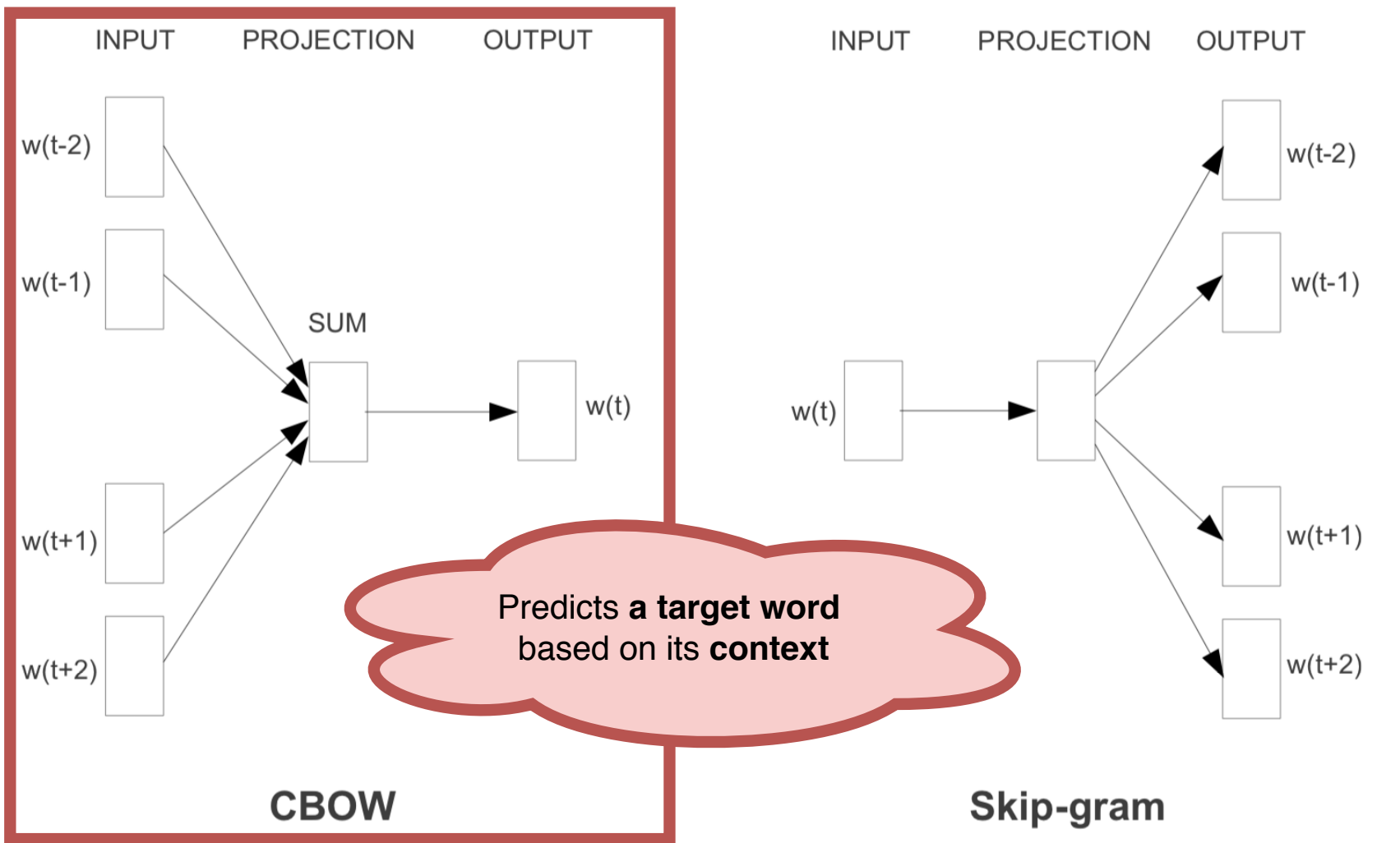
Predict between **every word** and its **context words**



Mikolov et al. (2013): Efficient Estimation of Word Representations in Vector Space. ICLR Workshop, 2013

word2vec

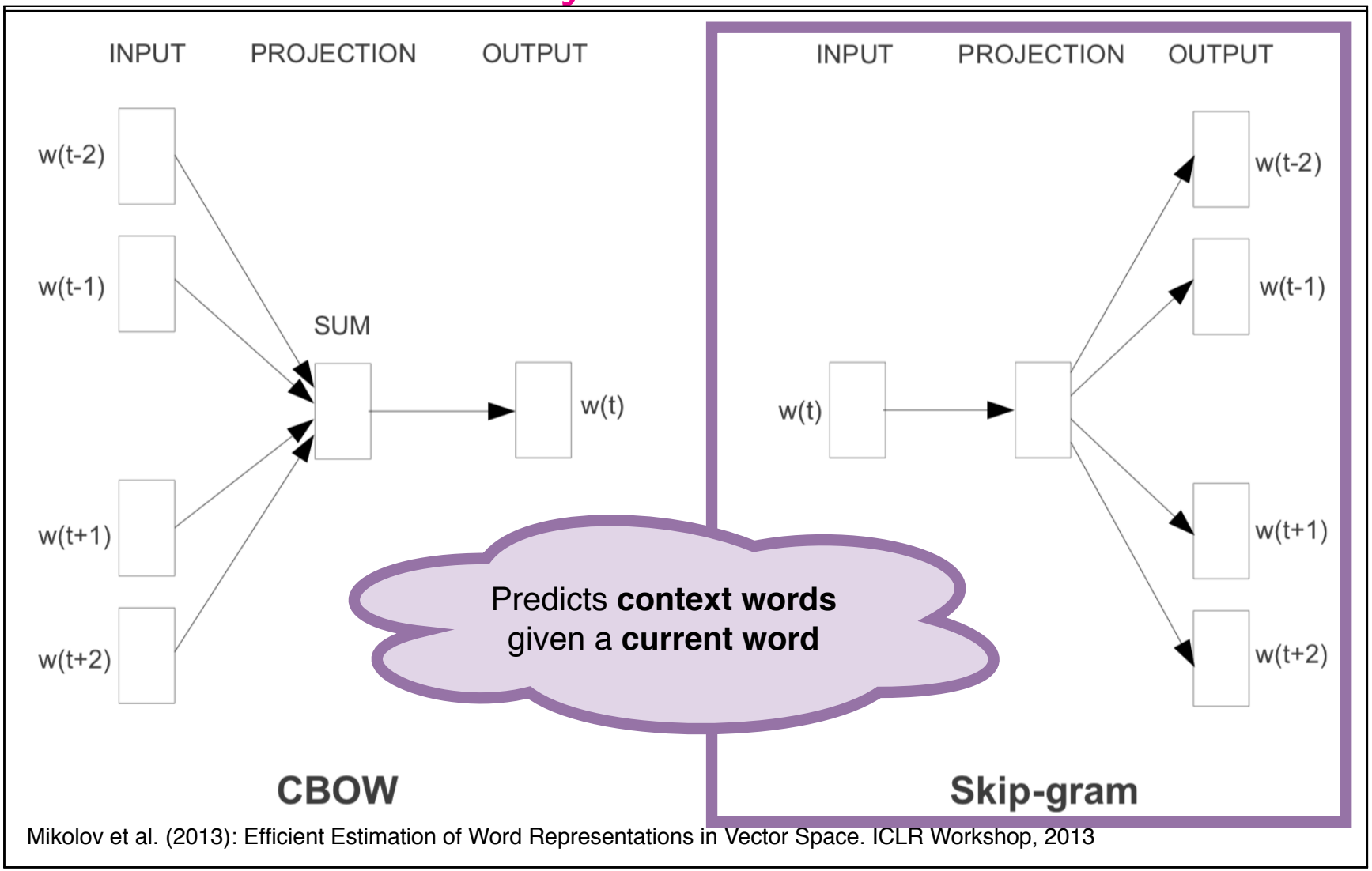
Predict between **every word** and its **context words**



Mikolov et al. (2013): Efficient Estimation of Word Representations in Vector Space. ICLR Workshop, 2013

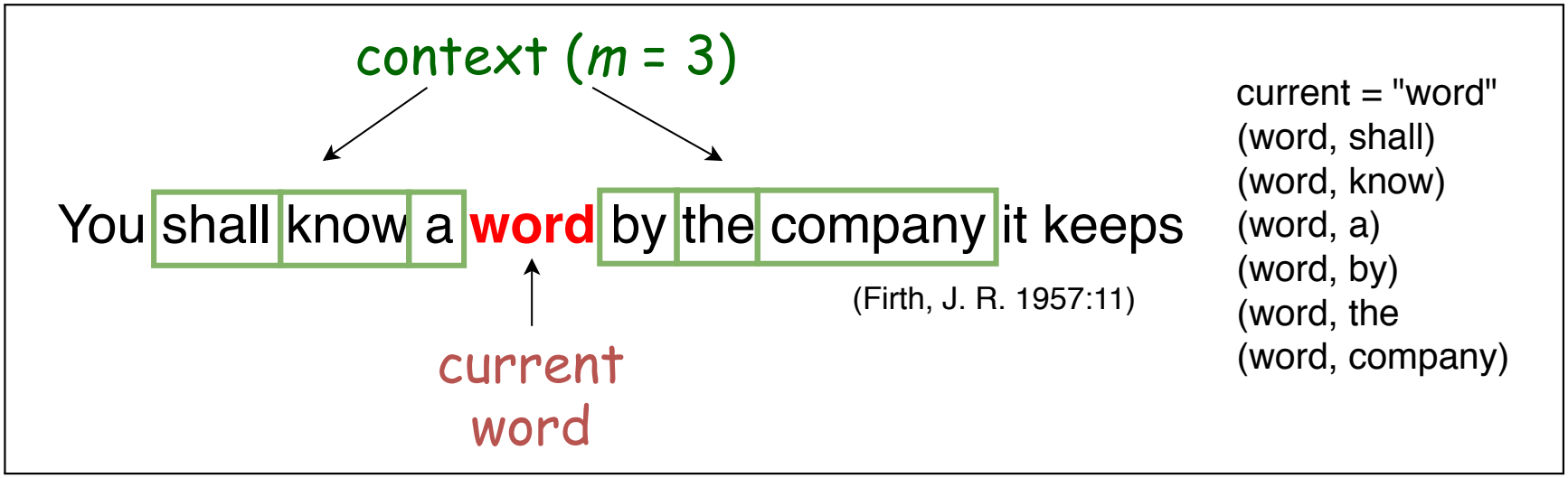
word2vec

Predict between **every word** and its **context words**

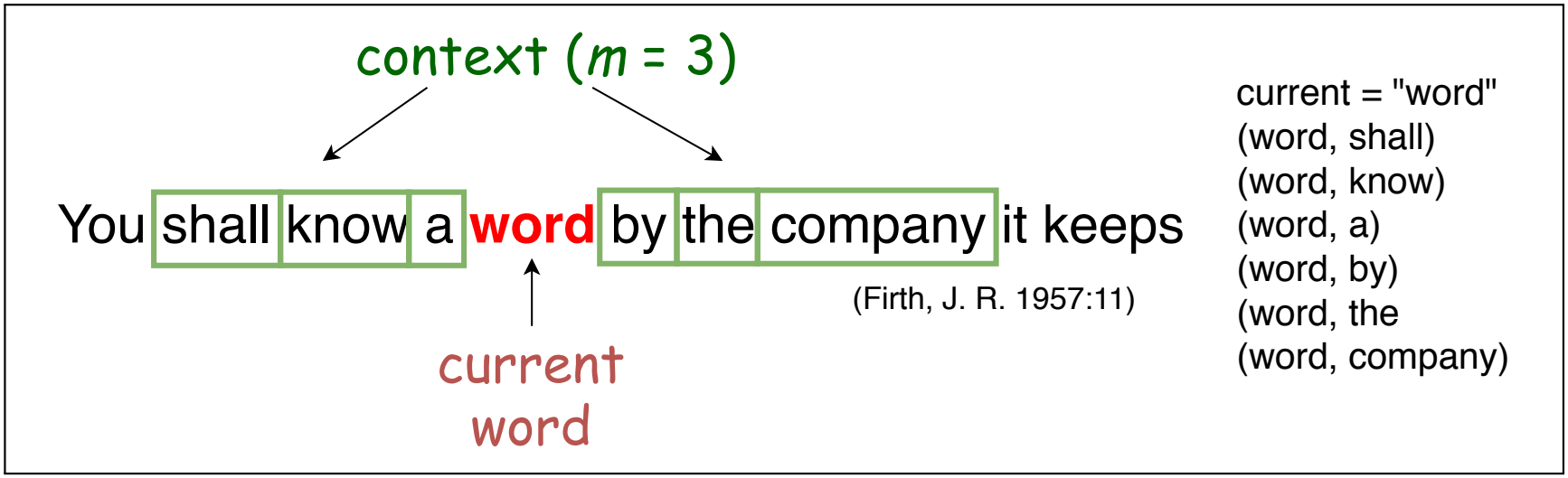


Mikolov et al. (2013): Efficient Estimation of Word Representations in Vector Space. ICLR Workshop, 2013

Skip-gram


context (m = 3)
You shall know a word by the company it keeps
(Firth, J. R. 1957:11)
current word
current = "word"
(word, shall)
(word, know)
(word, a)
(word, by)
(word, the
(word, company)

Skip-gram



Minimize:

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\leq j\leq m\\ i\neq 0}} \log p(w_{t+j}|w_t;\theta)$$

Skip-gram

$$p(w_{t+j}|w_t) = \frac{\exp({u_{w_{t+j}}}^\top v_{w_t})}{\sum_{w'=1}^{V} \exp({u_{w'}}^\top v_{w_t})}$$

- $\boldsymbol{u_w}$: vector representation of w as **context word**
- $\boldsymbol{v_w}$: vector representation of w as **current word**

Skip-gram

Softmax: from numbers to probability distributions

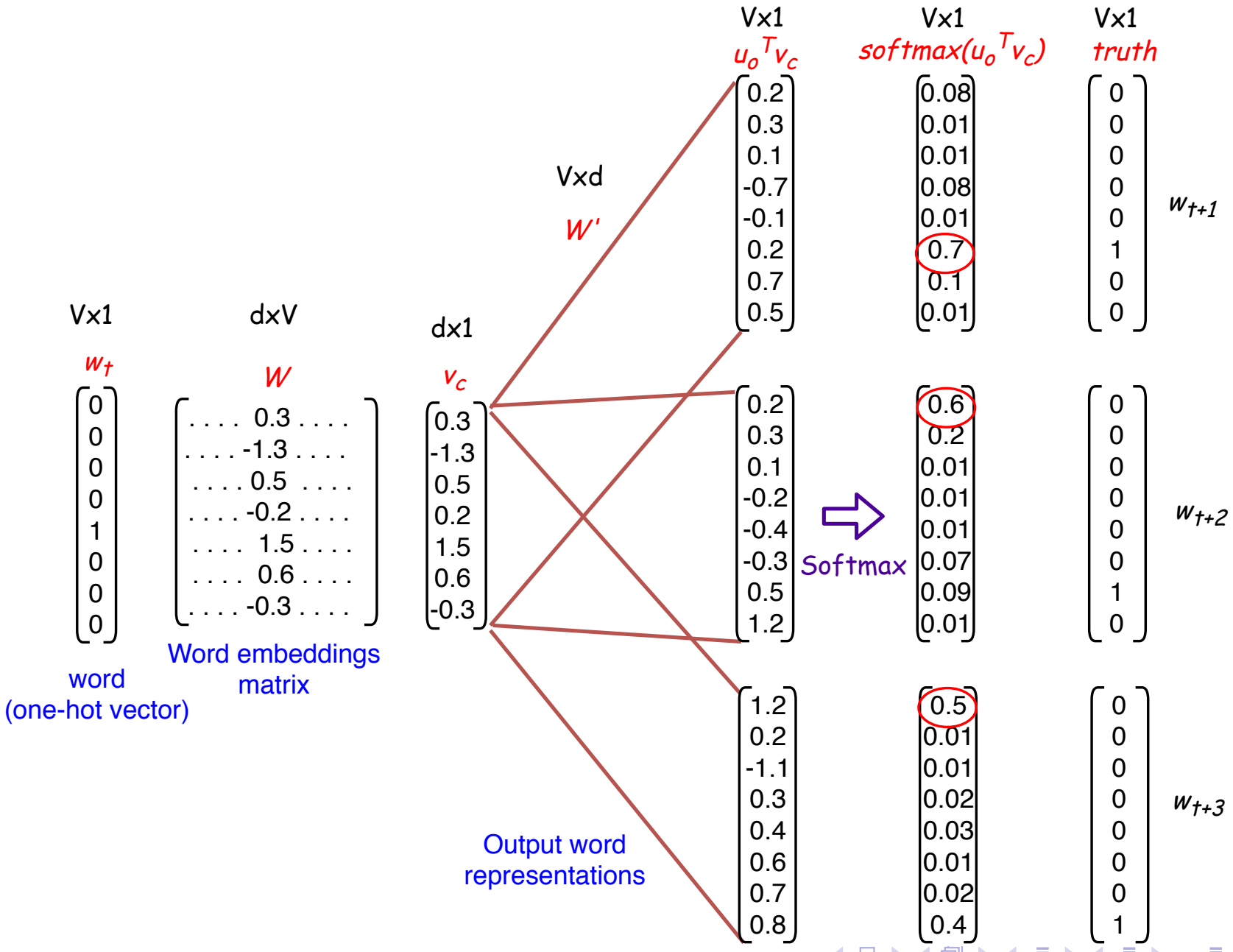
Exponentiate to make it positive

$$p(w_{t+j}|w_t) = \frac{\exp(u_{w_{t+j}}{}^\top v_{w_t})}{\sum_{w'=1}^{V} \exp(u_{w'}{}^\top v_{w_t})}$$

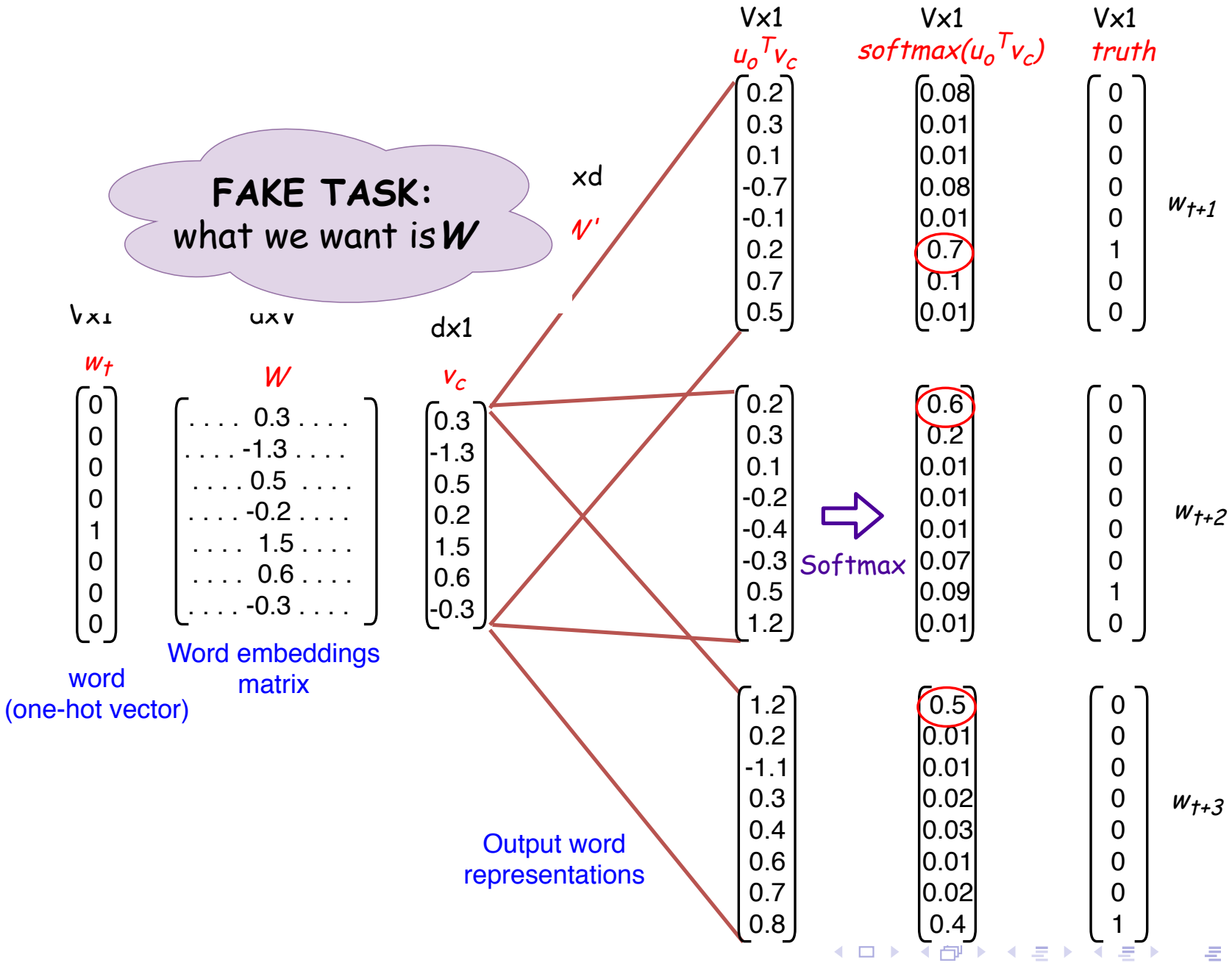
Normalised to give probability

- u_w: vector representation of w as **context word**
- v_w: vector representation of w as **current word**

Skip-gram


Vx1
w_t
0
0
0
0
1
0
0
0
word
(one-hot vector)
dxV
W
. . . . 0.3 . . . .
. . . . -1.3 . . . .
. . . . 0.5 . . . .
. . . . -0.2 . . . .
. . . . 1.5 . . . .
. . . . 0.6 . . . .
. . . . -0.3 . . . .
Word embeddings
matrix
dx1
v_c
0.3
-1.3
0.5
0.2
1.5
0.6
-0.3
Vxd
W'
Output word
representations
Vx1
$u_o{}^T v_c$
0.2
0.3
0.1
-0.7
-0.1
0.2
0.7
0.5
0.2
0.3
0.1
-0.2
-0.4
-0.3
0.5
1.2
1.2
0.2
-1.1
0.3
0.4
0.6
0.7
0.8
Softmax
Vx1
$softmax(u_o{}^T v_c)$
0.08
0.01
0.01
0.08
0.01
0.7
0.1
0.01
0.6
0.2
0.01
0.01
0.01
0.07
0.09
0.01
0.5
0.01
0.01
0.02
0.03
0.01
0.02
0.4
Vx1
truth
0
0
0
0
0
1
0
0
w_{t+1}
0
0
0
0
0
0
1
0
w_{t+2}
0
0
0
0
0
0
0
1
w_{t+3}

Skip-gram


FAKE TASK:
what we want is W
Vx1
w_t
0
0
0
0
1
0
0
0
word
(one-hot vector)
dxV
W
. . . . 0.3 . . . .
. . . . -1.3 . . . .
. . . . 0.5 . . . .
. . . . -0.2 . . . .
. . . . 1.5 . . . .
. . . . 0.6 . . . .
. . . . -0.3 . . . .
Word embeddings
matrix
dx1
v_c
0.3
-1.3
0.5
0.2
1.5
0.6
-0.3
xd
W'
Output word
representations
Vx1
$u_o{}^T v_c$
0.2
0.3
0.1
-0.7
-0.1
0.2
0.7
0.5
0.2
0.3
0.1
-0.2
-0.4
-0.3
0.5
1.2
1.2
0.2
-1.1
0.3
0.4
0.6
0.7
0.8
Softmax
Vx1
$softmax(u_o{}^T v_c)$
0.08
0.01
0.01
0.08
0.01
0.7
0.1
0.01
0.6
0.2
0.01
0.01
0.01
0.07
0.09
0.01
0.5
0.01
0.01
0.02
0.03
0.01
0.02
0.4
Vx1
truth
0
0
0
0
0
1
0
0
w_{t+1}
0
0
0
0
0
0
1
0
w_{t+2}
0
0
0
0
0
0
0
1
w_{t+3}

Skip-gram

- Training the model: compute **all vector gradients**
  - θ is a long **vector of word vectors**
  - each word has **2 vectors**

$$\theta = \begin{bmatrix} v_{aardvak} \\ v_a \\ \dots \\ v_{zebra} \\ u_{aardvak} \\ u_a \\ \dots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

Skip-gram

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\leq j\leq m\\ i\neq 0}} \log p(w_{t+j}|w_t;\theta)$$

- How to **minimise** $J(\theta)$? $\rightarrow$ **(stochastic) gradient decent**
  - Calculate **gradients for each** v **and each** u on a window

- **Examples:** gradient of the log likelihood w.r.t. v_{w_t}

$$\frac{\partial \log p(w_{t+j}|w_t)}{\partial v_{w_t}} = u_{w_{t+j}} - \mathop{\mathbb{E}}_{x\sim p(x|w_t)}[u_x]$$

Skip-gram

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\leq j\leq m\\ i\neq 0}} \log p(w_{t+j}|w_t;\theta)$$

- How to **minimise** $J(\theta)$? $\rightarrow$ **(stochastic) gradient decent**
  - Calculate **gradients for each** v **and each** u on a window
- **Examples:** gradient of the log likelihood w.r.t. v_{w_t}

$$\frac{\partial \log p(w_{t+j}|w_t)}{\partial v_{w_t}} = u_{w_{t+j}} - \mathop{\mathbb{E}}_{x\sim p(x|w_t)}[u_x]$$

Gradient calculations are left for you!

Christopher Manning: Natural Language Processing with Deep Learning @ Stanford University:
https://www.youtube.com/watch?v=ERibwqs9p38

Skip-gram

Problem: computing a single forward pass $\rightarrow$ **sum across the entire vocabulary**

- Example: for **300 dimensions** and **V = 10K words**
  - when training for each pair (w_t, w_{t+j})
  - **3M weights** in each the hidden layer and output layer

Solution: perform approximations!

- Hierarchical softmax
- **Negative sampling**

Negative sampling

- Modify the weights only for **a small sample of words**

Negative sampling

- Modify the weights only for **a small sample of words**
- **Negative sample:** words that **are not part of the context**

Negative sampling

- Modify the weights only for **a small sample of words**
- **Negative sample:** words that **are not part of the context**
- Change the task: **binary logistic regression classifiers**

I want a glass of orange juice to go along with my cereal.

	x		y
	context	current word	target
	orange	**juice**	**1**
K	orange	king	0
	orange	book	0
	orange	the	0
	orange	of	0

Negative sampling

- Modify the weights only for **a small sample of words**
- **Negative sample:** words that **are not part of the context**
- Change the task: **binary logistic regression classifiers**

I want a glass of orange **juice** to go along with my cereal.

	x: context	x: current word	y: target
	orange	**juice**	**1**
k	orange	king	0
k	orange	book	0
k	orange	the	0
k	orange	of	0

Minimize:

Approximate $\mathbb{E}$ by drawing **sample of size** ***k*** from **noise distribution**

$$-\left(\log Q_\theta(y=1|w_{t+j}, w_t) + k \mathop{\mathbb{E}}_{\tilde{w}\sim P_{noise}} [\log Q_\theta(y=0|w_{t+j}, \tilde{w})]\right)$$

Probability of pair (w_{t+j}, w_t) to be **positive** calculated **according to** θ

Negative sampling

- Modify the weights only for **a small sample of words**
- **Negative sample:** words that **are not part of the context**
- Change the task: **binary logistic regression classifiers**

I want a glass of orange juice to go along with my cereal.

	x: context	x: current word	y: target
	orange	**juice**	**1**
K	orange	king	0
K	orange	book	0
K	orange	the	0
K	orange	of	0

For $k = 4$
- update positive word ("juice")
- update 4 negative words
- output: 5 neurons, **1,500 weight values**
(0.05% of 3M)

Minimize:

Approximate $\mathbb{E}$ by drawing **sample of size k** from **noise distribution**

$$-\left(\log Q_\theta(y=1|w_{t+j}, w_t) + k \mathop{\mathbb{E}}_{\tilde{w}\sim P_{noise}} [\log Q_\theta(y=0|w_{t+j}, \tilde{w})]\right)$$

Probability of pair (w_{t+j}, w_t) to be **positive** calculated **according to θ**

Why does it work?

- Let's assume that the word representations are **organised semantically**
- words w_1 and w_2 having similar meaning would be **close to each other** in this space

  $\rightarrow$ Consequently $\mathcal{F}(w_1) \approx \mathcal{F}(w_2)$

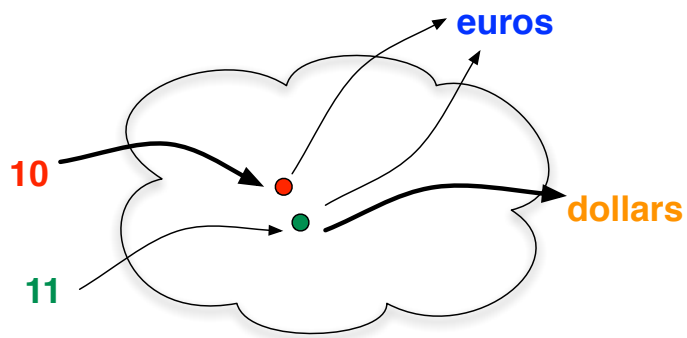
Why does it work?

- Let's assume that the word representations are **organised semantically**
- words w_1 and w_2 having similar meaning would be **close to each other** in this space

  $\rightarrow$ Consequently $\mathcal{F}(w_1) \approx \mathcal{F}(w_2)$

Language modelling:

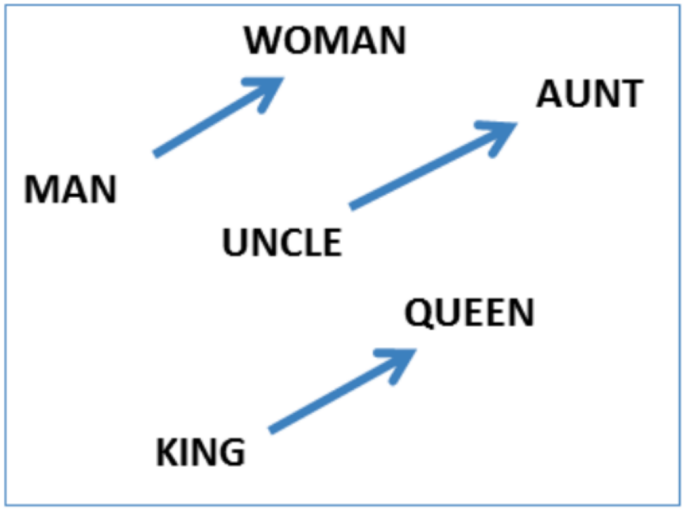
1. I have got **10** euros in my wallet
2. This item costs 11 euros
3. In the U.S. it is 11 **dollars** !



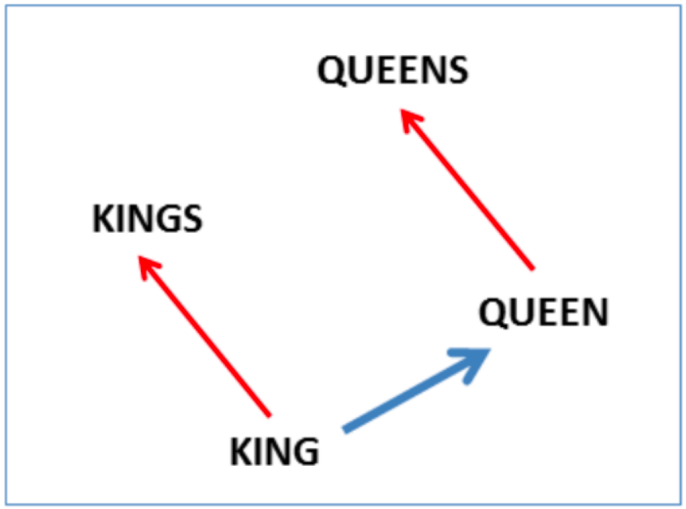
$\Rightarrow$ What is the probability that **10** is followed by **dollars**?

What can we do with word2vec?

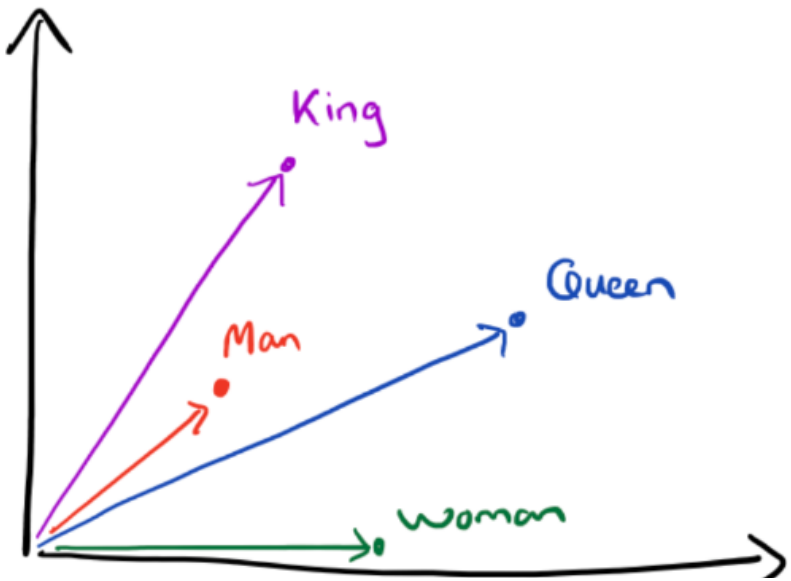
Relationships

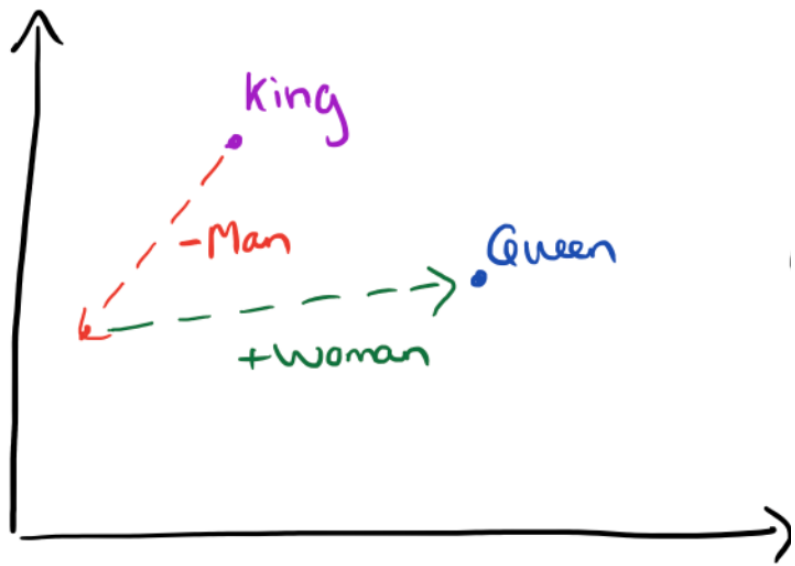


Plural



General semantics: king - man + woman = queen

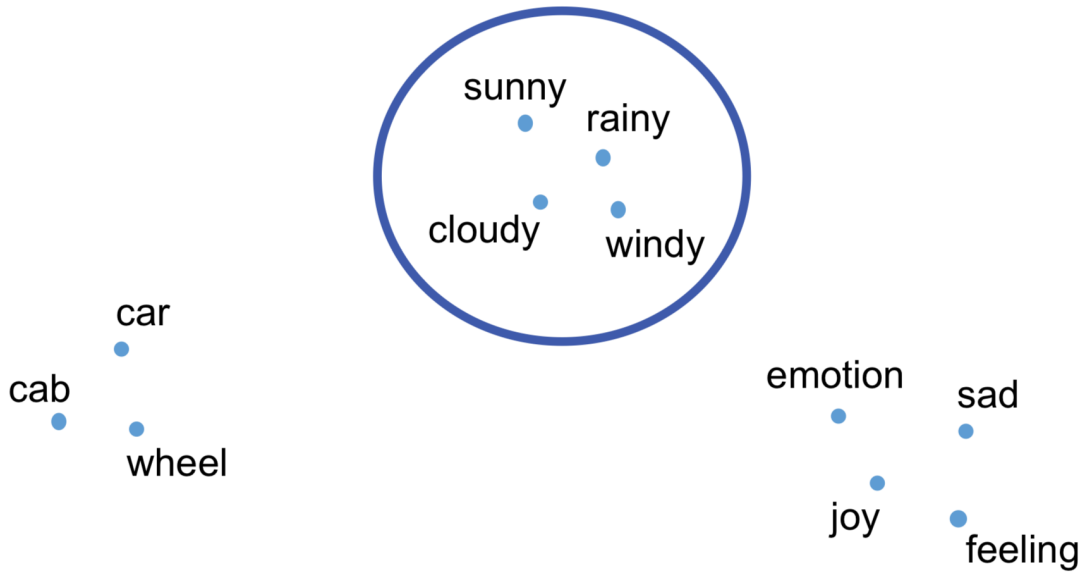




https://blog.acolyer.org/2016/04/21/the-amazing-power-of-word-vectors/

What can we do with word2vec?

Good to find **some relationship** between words



Kai-Wei Chang @ University of Virginia: http://www.cs.virginia.edu/~kc2wc/teaching/NLP16/slides/07-WordEmbedding.pdf

Text Processing: Deep Learning: Overview

- Shortest introduction to Neural networks
- Representing words
- **Representing sentences**
- Deep Learning for Sentiment Analysis
- Deep Learning for Information Extraction (NER)

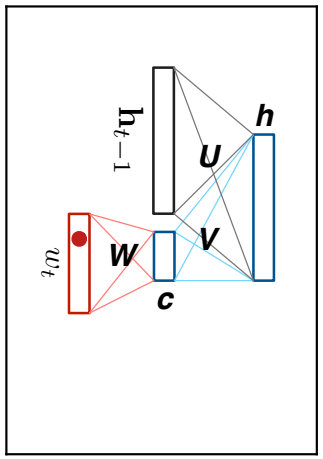
How to represent sentences?

Sentence = sequence of word $\Rightarrow$ we need an **encoder**
Several possibilities have been developed:

How to represent sentences?

Sentence = sequence of word $\Rightarrow$ we need an **encoder**
Several possibilities have been developed:

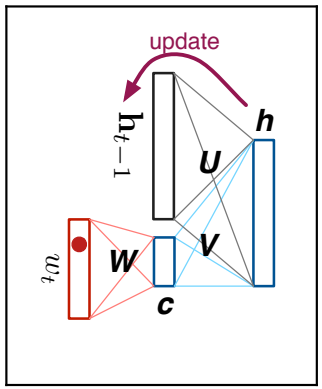
- **Recurrent neural network** (RNN)
  - and its **bidirectional** version
  - representation = single vector or matrix



How to represent sentences?

Sentence = sequence of word $\Rightarrow$ we need an **encoder**
Several possibilities have been developed:

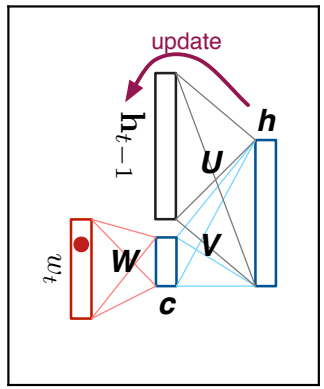
- **Recurrent neural network** (RNN)
  - and its **bidirectional** version
  - representation = single vector or matrix



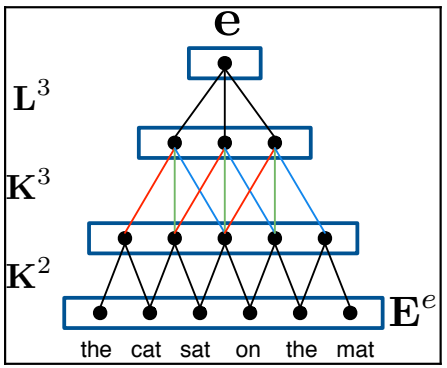
How to represent sentences?

Sentence = sequence of word $\Rightarrow$ we need an **encoder**
Several possibilities have been developed:

- **Recurrent neural network** (RNN)
  - and its **bidirectional** version
  - representation = single vector or matrix



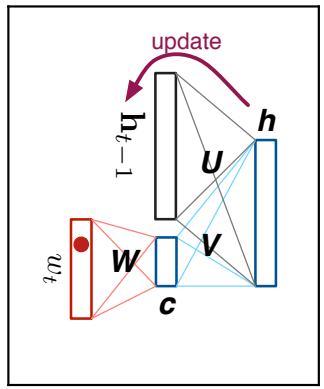
- **Convolutional neural network** (CNN)
  - produces a single vector representation



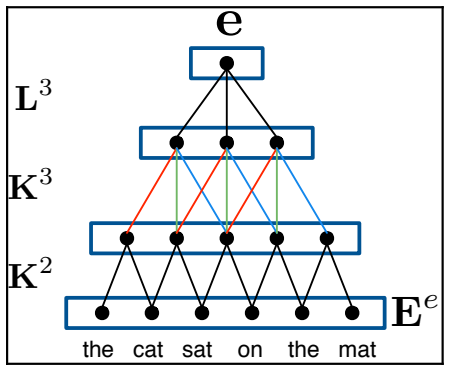
How to represent sentences?

Sentence = sequence of word $\Rightarrow$ we need an **encoder**
Several possibilities have been developed:

- **Recurrent neural network** (RNN)
  - and its **bidirectional** version
  - representation = single vector or matrix

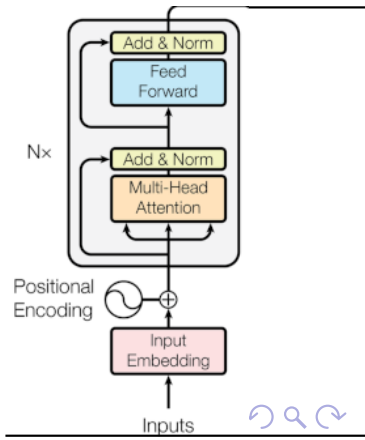


- **Convolutional neural network** (CNN)
  - produces a single vector representation



- Very recently **Transformers** = self-attention
  - representation = matrix (1 vector per word)
  - Must read:

    `http://jalammar.github.io/illustrated-transformer/`



How to classify sentences?

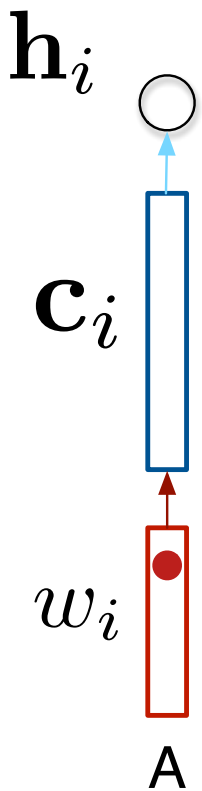
- The classifier is a neural network implementing a complex function $\mathcal{F}$
  - that operates in the **continuous space**
  - that maps input vectors to a **probability distribution** over the desired classes

1. Encode the sentence
   - get a vector
   - get a matrix (1 vector per word) $\rightarrow$ compress into 1 vector
     - **pooling** operation (usually mean or max)
     - concatenation
2. Non-linear classification layer(s) $\rightarrow$ get a vector of scores $\mathbf{z}$ (1 for each class)
3. Get a probability distribution by normalization $\rightarrow$ softmax

$$p(\mathbf{c} = j|\theta) = \frac{e^{\mathbf{z}_j}}{\sum_{k=1}^{\|V\|} e^{\mathbf{z}_k}}$$

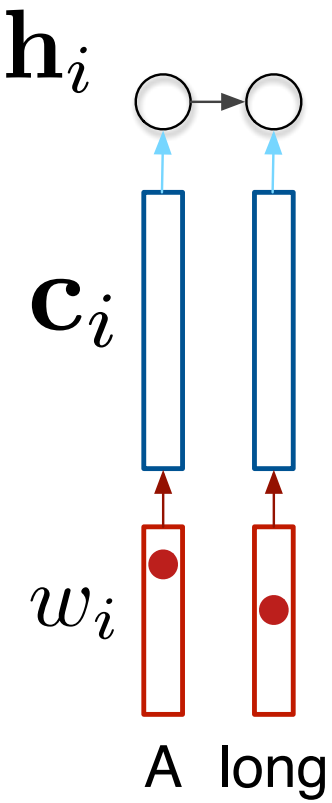
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"



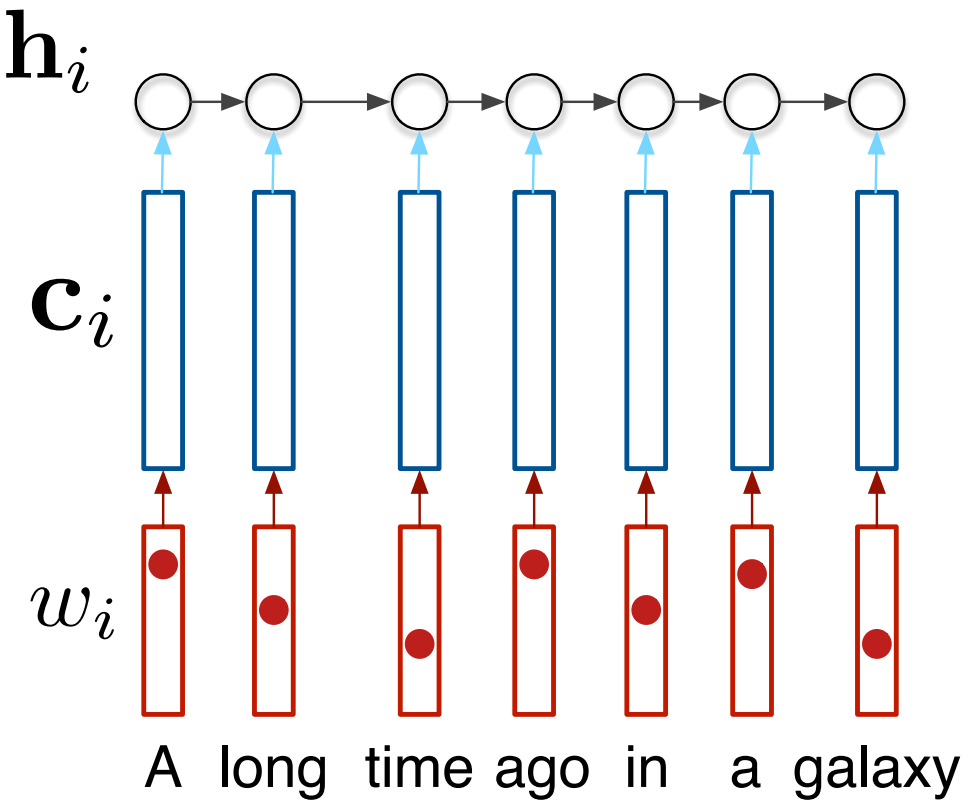
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"



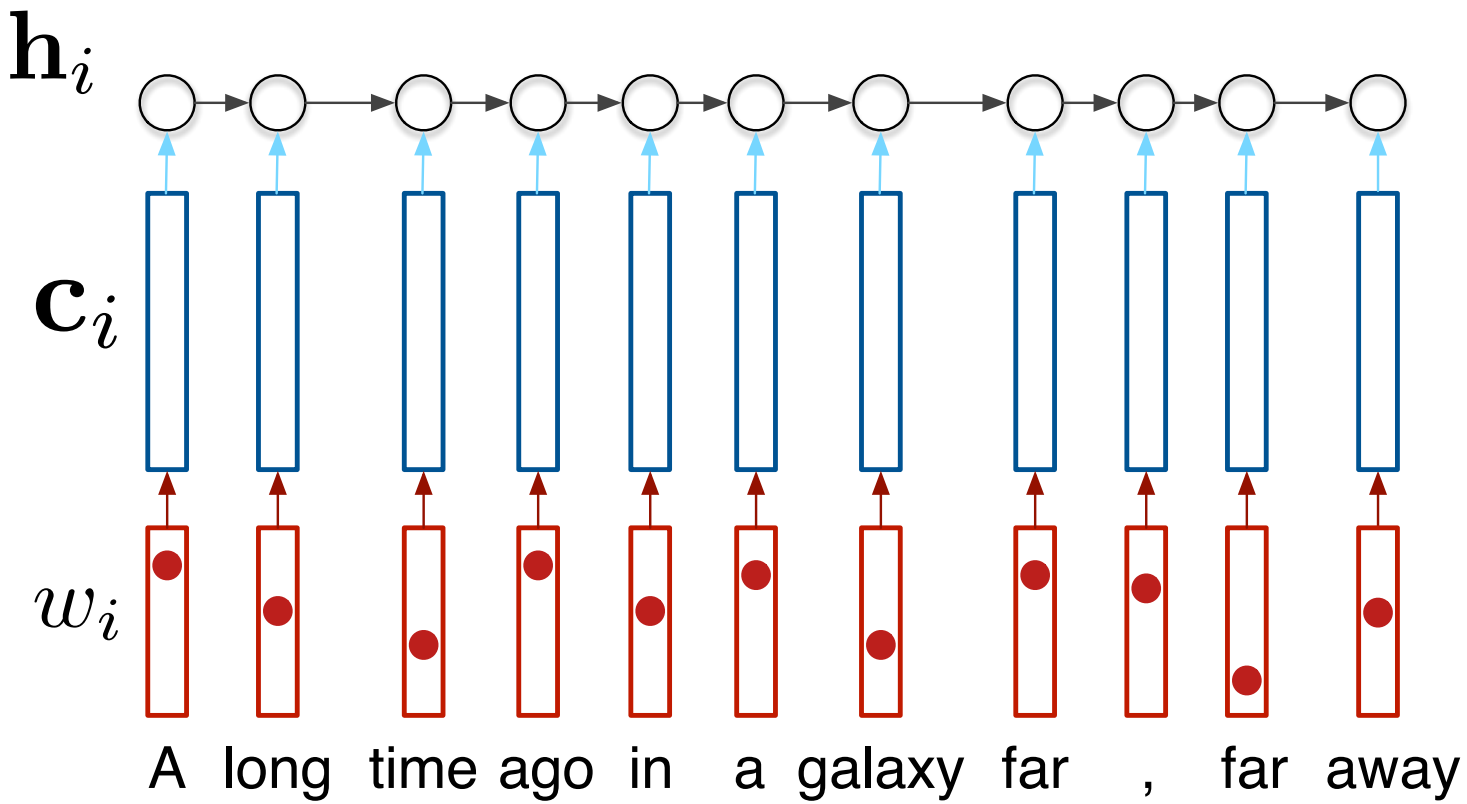
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"



Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"



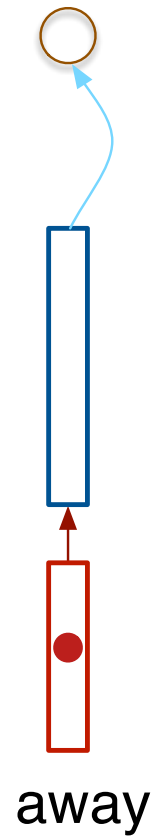
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"

$\mathbf{h}_i$

$\mathbf{c}_i$

w_i



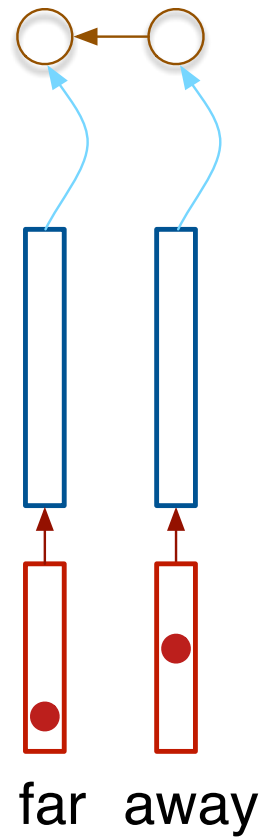
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"

$\mathbf{h}_i$

$\mathbf{c}_i$

w_i



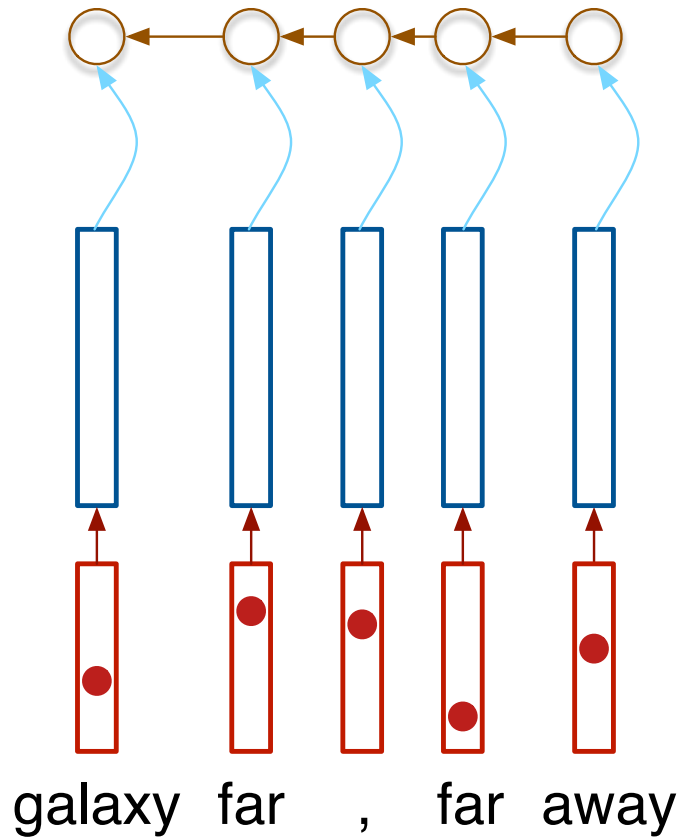
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"

$\mathbf{h}_i$

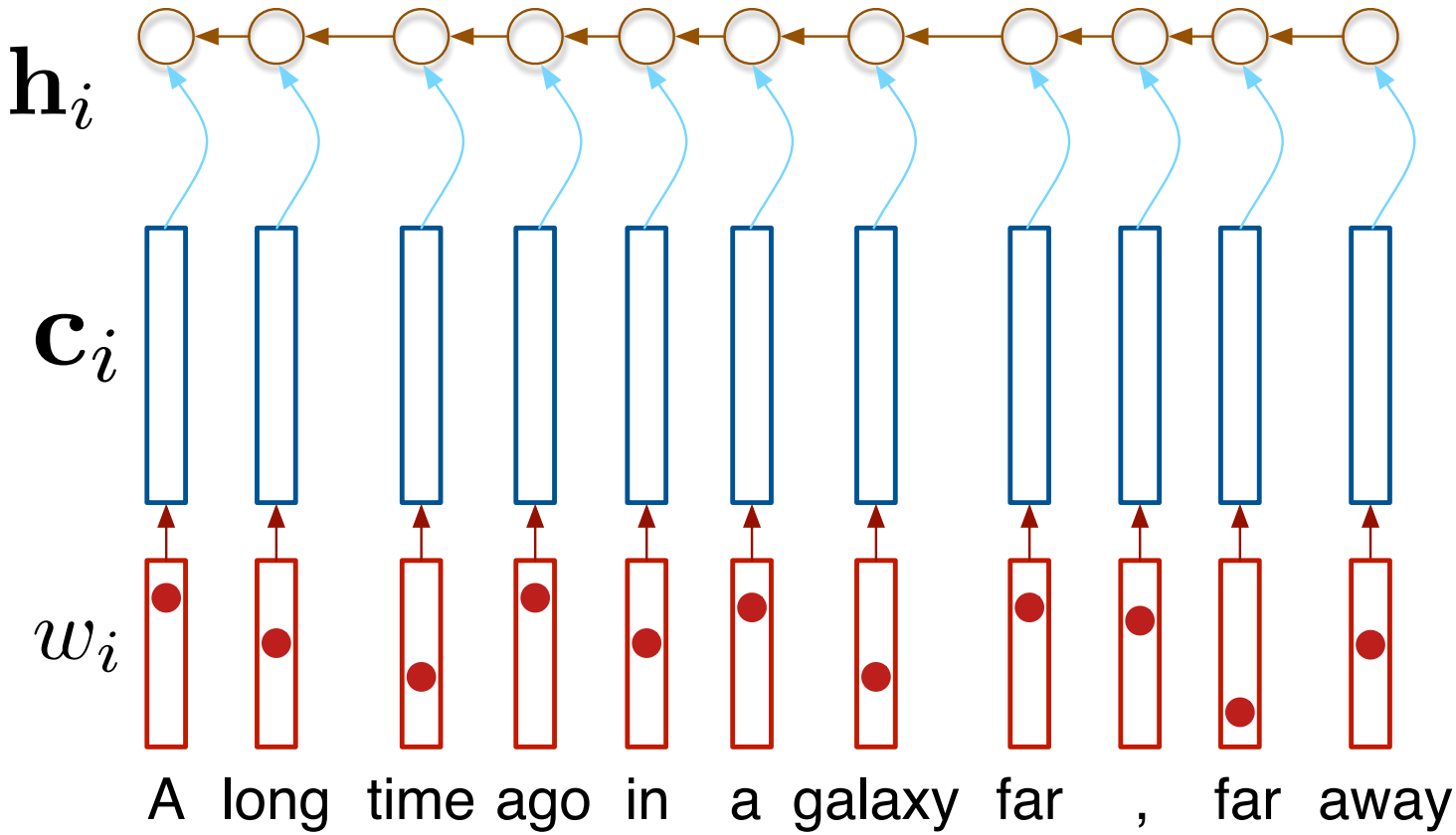
$\mathbf{c}_i$

w_i



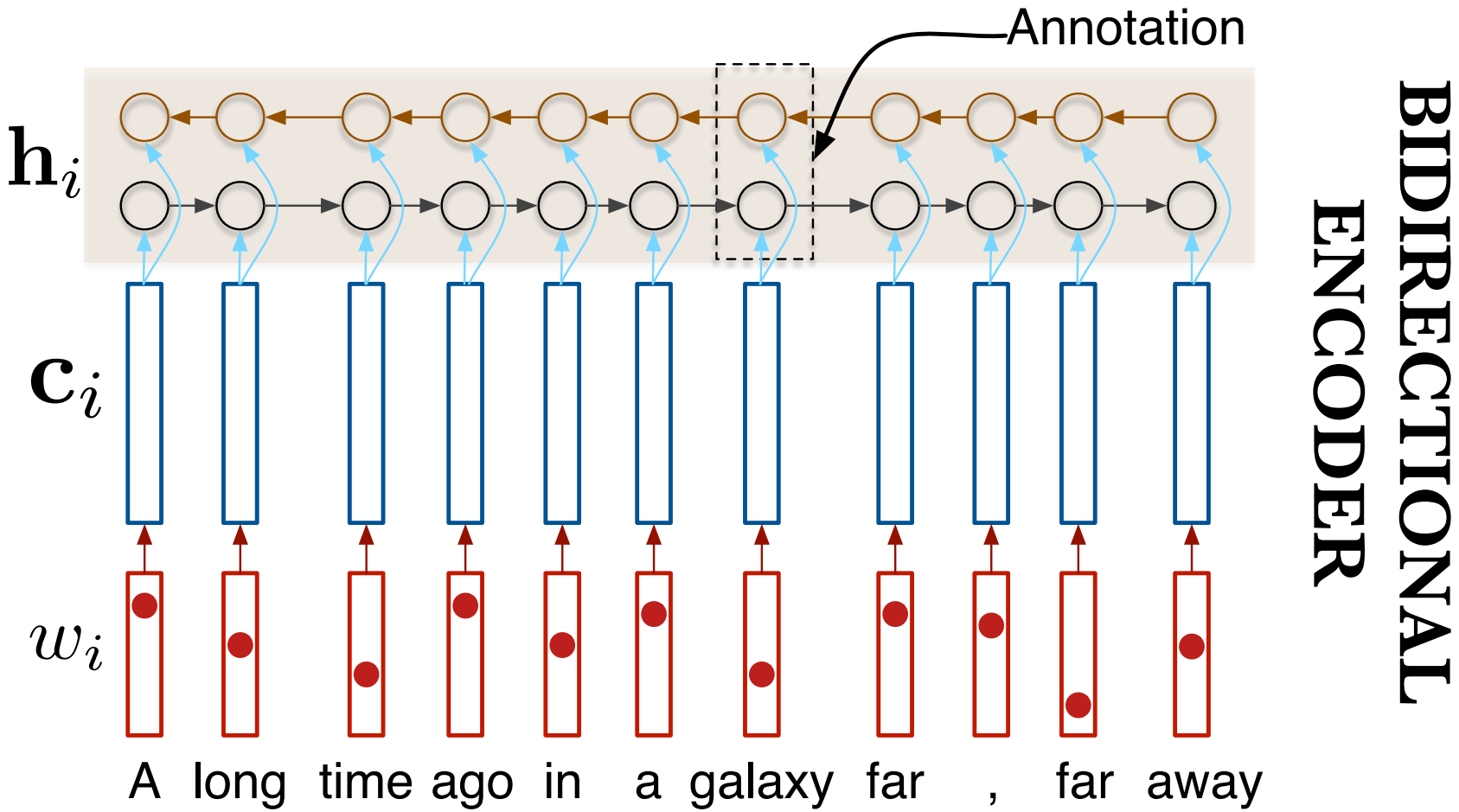
Encoding a sentence with a (bi-)RNN

Sentence: "**A long time ago in a galaxy far , far away**"



Encoding a sentence with a (bi-)RNN

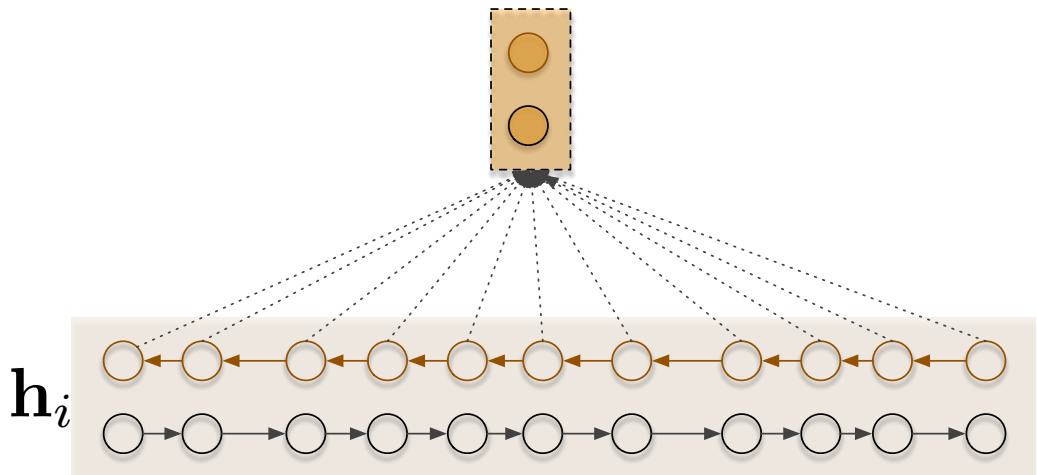
Sentence: "**A long time ago in a galaxy far , far away**"



Pooling operation

Compute the feature-wise **average** or **maximum** **activation** of a set of vectors

Aim: sub-sampling $\rightarrow$ result is a vector!



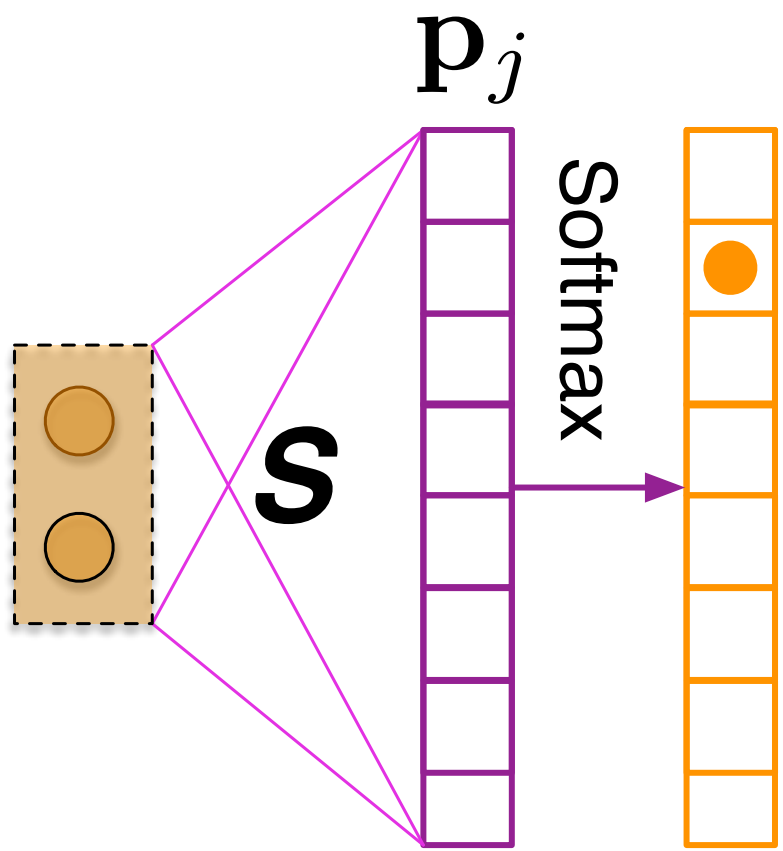
[Source: A comment on max pooling to read:

`https://mirror2image.wordpress.com/2014/11/11/geoffrey-hinton-on-max-pooling-reddit-ama/`]

Classification layer $\Rightarrow$ Softmax

Get a probability distribution by normalization $\rightarrow$ softmax:

$$p(\mathbf{c} = j|\theta) = \frac{e^{\mathbf{z}_j}}{\sum_{k=1}^{\|V\|} e^{\mathbf{z}_k}}$$



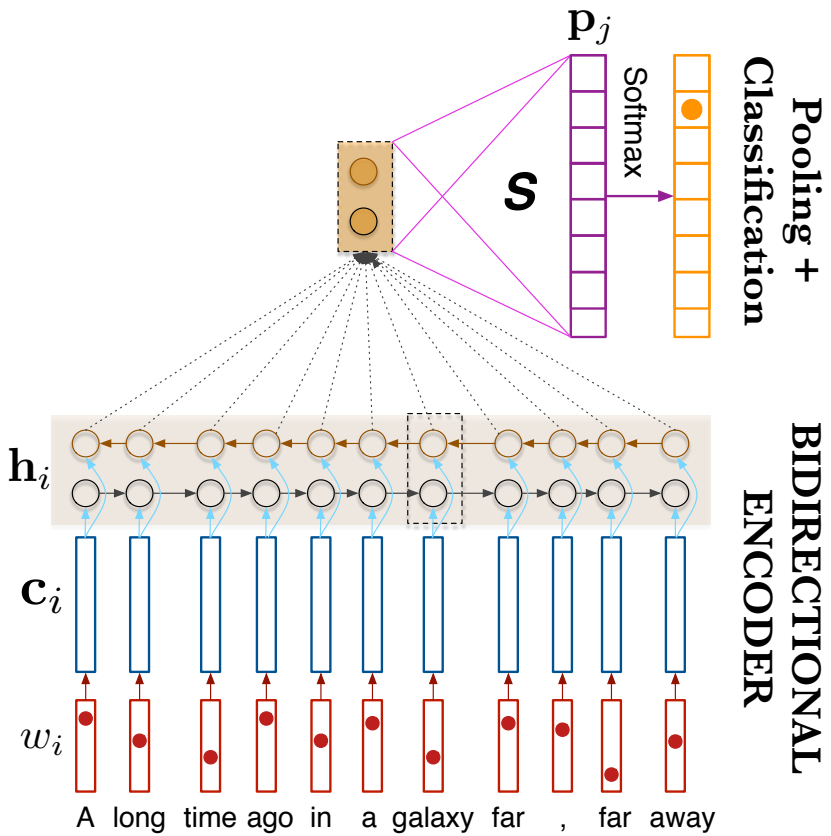
Text Processing: Deep Learning: Overview

- Shortest introduction to Neural networks
- Representing words
- Representing sentences
- **Deep Learning for Sentiment Analysis**
- **Deep Learning for Information Extraction (NER)**

Deep Learning for Sentiment Analysis

Principle

Project or represent the **text** into a **continuous space** and train an estimator operating into this space to compute the probability of the sentiment.

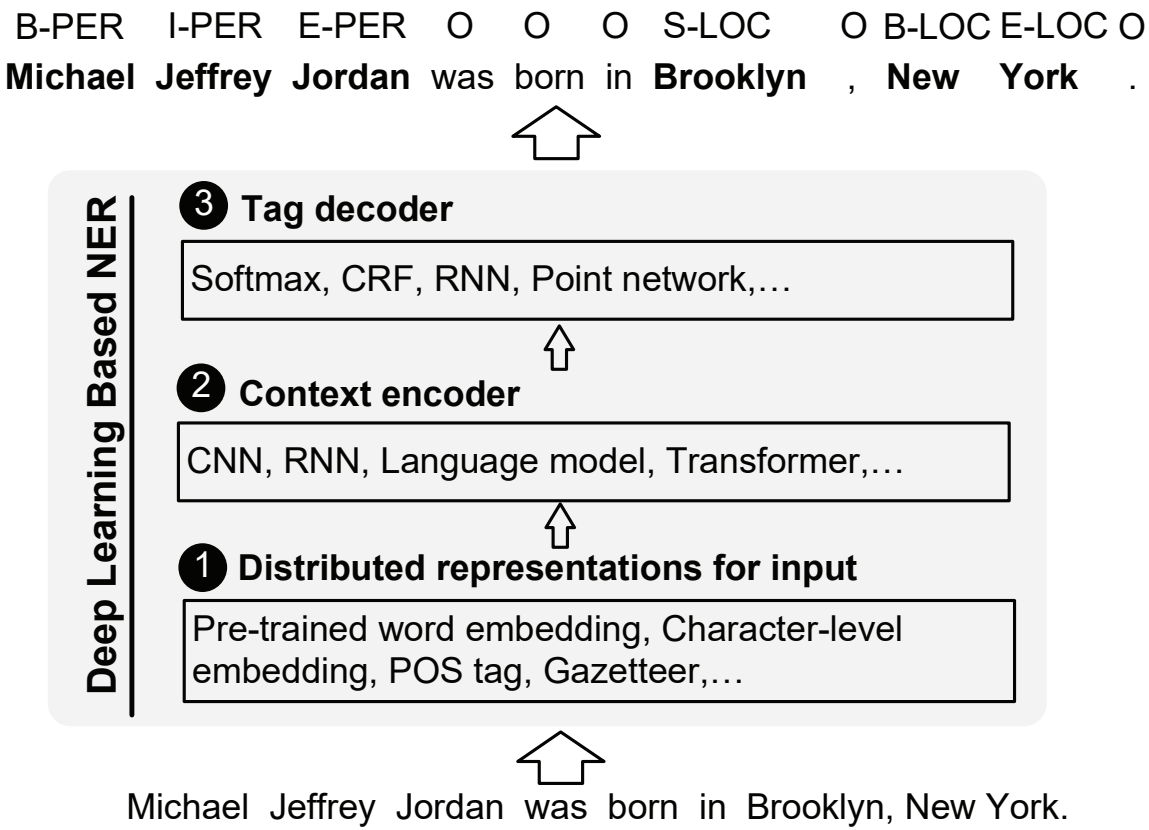


Deep Learning for SA

- Sahoo, C., Wankhade, M. & Singh, B.K. Sentiment analysis using deep learning techniques: a comprehensive review. Int J Multimed Info Retr 12, 41 (2023) https://doi.org/10.1007/s13735-023-00308-2 [Sahoo et al., 2023]

Deep Learning for NER

- Li, J., Sun, A., Han, J., & Li, C. (2018).
  A Survey on Deep Learning for Named Entity Recognition.
  `http://arxiv.org/abs/1812.09449` [Li et al., 2018]



Reading material

- Mikolov et al. (2013)
  Distributed Representations of Words and Phrases and their Compositionality.
  NeurIPS 2013.
  `https://arxiv.org/pdf/1310.4546.pdf`

What I couldn't talk about today

- **GloVe:** Global Vectors for Word Representation [Pennington et al., 2014]
- Recurrent cells addressing the problem of **vanishing gradient**
  - **LSTM:** Long Short-Term Memory [Hochreiter and Schmidhuber, 1997]
  - **GRU:** Gated Recurrent Unit [Cho et al., 2014]
- **Transformer models** → *Attention is All you Need* [Vaswani et al., 2017]
- **Contextualised word embeddings:**
  - **ELMo:** Embeddings from Language Models [Peters et al., 2018]
  - **BERT:** Bidirectional Encoder Representations from Transformers [Devlin et al., 2018]
  - **XLNet** [Yang et al., 2019]
- Large Language Models → **GPT** [Radford et al., 2018, Brown et al., 2020]

Text Processing: Deep Learning: Resources

- Deep Learning book:
  `https://www.deeplearningbook.org/`
  [Goodfellow et al., 2016]

- Set of notebooks for Sentiment Analysis in Pytorch
  `https://github.com/bentrevett/pytorch-sentiment-analysis`

- Enjoy the lectures COM4513/6513 Natural Language Processing - Spring 21-22

Credit: slides based on Dr Loïc Barrault's slides for the autumn 2020/2021 course.

References I

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020).

Language models are few-shot learners.

Proceedings of the 2020 NeurIPS.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014).

Learning phrase representations using RNN encoder-decoder for statistical machine translation.

In *EMNLP 2014 - 2014 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*.

Devlin, J., Ming-Wei, Lee, C. K., and Toutanova, K. (2018).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

CoRR, arXiv:1810.04805.

References II

Goodfellow, I., Bengio, Y., and Courville, A. (2016).

Deep Learning.

MIT Press.

http://www.deeplearningbook.org.

Hochreiter, S. and Schmidhuber, J. (1997).

Long Short-Term Memory.

Neural Computation.

Li, J., Sun, A., Han, J., and Li, C. (2018).

A Survey on Deep Learning for Named Entity Recognition.

Pennington, J., Socher, R., and Manning, C. D. (2014).

Glove: Global vectors for word representation.

Proceedings of the 2014 EMNLP, pages 1532–1543.

Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. (2018).

Deep contextualized word representations.

Proceedings of the 2018 NAACL.

References III

Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. (2018).

Improving language understanding by generative pre-training.

pre-print.

Sahoo, C., Wankhade, M., and Singh, B. (2023).

Sentiment analysis using deep learning techniques: a comprehensive review.

Int J Multimed Info Retr, 12(41):342–351.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017).

Attention is all you need.

In *Advances in Neural Information Processing Systems*.

Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., and Le, Q. V. (2019).

Xlnet: Generalized autoregressive pretraining for language understanding.

Proceedings of the 2019 NeurIPS.